

booster

Booster T1 多机器人足球协同决策系统：环境准备与完成路线

本文档根据本文件夹中的《软件工程短学期作业要求.docx》、当前仓库代码，以及 Booster 官网开源资料整理。**Booster T1 多机器人足球协同决策系统。**

1. 任务目标

课程要求的核心目标不是只写一个能跑的程序，而是用软件工程方法完成一个可协作、可运行、可测试、可展示、可复现的机器人足球决策系统。

题目四的最低验收标准是：

- 两个机器人能够基于统一 WorldState 完成一次有效传球，或完成一次带球推进。
- 系统能够输出完整的决策过程日志。
- 需要提交协作策略说明、状态机或行为树图、策略接口说明、场景测试报告和多机器人协作演示视频。

我们当前仓库已经实现了题目四的基础骨架：

- common/world_state.py：统一世界状态。
- common/robot_action.py：机器人动作接口。
- strategy/：传球、带球、射门、跑位、卡位策略。
- decision/decision_fsm.py：决策状态机。
- simulation/field_simulator.py：自包含 2D mock 足球仿真。
- tests/：单元测试和集成测试。
- main.py：演示入口，支持 mock 和 real 两种模式。

2. Booster 官方资料与我们项目的关系

Booster 官网的开源资料页面给出了几个关键方向：

- Booster Robotics SDK：面向 Booster 机器人二次开发的 SDK，支持 C++ 和 Python。
- ROS2 SDK：提供机器人控制相关的 ROS2 消息和服务定义。
- RoboCup Demo：覆盖感知、定位、决策、执行的 RoboCup 3v3 示例。
- Booster Gym / Booster Train / Booster Deploy / Booster Assets：分别偏强化学习训练、Isaac Lab 训练、部署和模型资源。

对我们题目四来说，最相关的是：

官方资料	和本项目的关系
Booster Robotics SDK	后续接实机或官方接口时，用于控制机器人或读取状态。
ROS2 SDK	如果和其他组、仿真平台或真实机器人通过 ROS2 联调，需要使用它的消息和服务定义。
RoboCup Demo	可以参考其 vision / brain / game_controller 分层思路，其中 brain 对应我们的策略决策模块。

官方资料	和本项目的关系
Booster Assets	获取 Booster 机器人模型、数据或资源，用于仿真/展示扩展。

目前建议不要一开始就把官方 RoboCup Demo 全量并入我们仓库。它是完整比赛级工程，复杂度高。更务实的路线是：先把我们自己的 mock 决策系统做稳定，再逐步把 WorldState 和 RobotAction 接到 Booster 官方 SDK、ROS2 SDK 或其他小组提供的仿真平台。

3. 电脑需要具备的环境

3.1 基础环境：完成当前仓库必须具备

这部分用于运行当前 `project4_multi_agent` 仓库。

必须安装：

- Git：用于多人协作、拉取、提交和推送代码。
- Python 3.8 或更高版本：当前电脑是 Python 3.11，可以使用。
- pip：安装测试和可视化依赖。
- VS Code 或 PyCharm：用于编辑代码。

建议安装：

- pytest：运行测试。
- matplotlib：当前电脑已经安装，用于可视化。
- Git Bash 或 WSL2：如果要直接运行 `run.sh`，Windows PowerShell 不适合直接执行 bash 脚本。

当前电脑检查结果：

- Python 已安装：Python 3.11.0
- matplotlib 已安装。
- pytest 未安装。
- 当前项目可以运行 mock 演示，但 Windows 默认 GBK 编码会导致 emoji 输出报错，需要启用 UTF-8。

3.2 进阶环境：接入 Booster 官方仿真/ROS2/实机时需要

如果我们只做课程题目四的基础验收，当前 Python mock 仿真已经够用。如果要和 Booster 官方资料、MuJoCo、ROS2 或其他小组联调，建议准备 Linux 环境。

推荐方案：

- Windows 电脑保留当前项目开发。
- 安装 WSL2 + Ubuntu 22.04 LTS。
- 在 Ubuntu 中安装 ROS2 Humble、colcon、CMake、gcc/g++、Python 开发工具。
- 在 Ubuntu 中再克隆 Booster 官方 SDK、ROS2 SDK、RoboCup Demo。

原因：

- Booster Robotics SDK 官方预构建环境写的是 Ubuntu 22.04 LTS，CPU 支持 aarch64 和 x86_64，编译器 gcc 11.4.0。

- ROS2 Humble 与 Ubuntu 22.04 匹配度最高。
- RoboCup Demo 的构建脚本是 Linux shell 脚本，在 Windows 下容易遇到换行符问题。

4. 基础环境安装步骤

4.1 安装 Git

下载安装 Git for Windows:

```
winget install --id Git.Git -e
```

安装后检查:

```
git --version
```

配置提交用户名和邮箱。邮箱建议使用 GitHub 账号验证过的邮箱:

```
git config --global user.name "我们的名字或GitHub用户名"  
git config --global user.email "我们的GitHub邮箱"
```

4.2 配置 GitHub SSH

我们已经在做这一步。最终目标是让下面命令成功:

```
ssh -T git@github.com
```

成功时通常会看到类似:

```
Hi 用户名! You've successfully authenticated...
```

注意:

- .pub 后缀文件是公钥，可以添加到 GitHub。
- 没有 .pub 后缀的是私钥，不能发给任何人，也不能提交进仓库。
- 当前仓库远程地址应为:

```
git remote -v
```

应看到:

```
origin  git@github.com:Ouy5517/project4_multi_agent.git (fetch)  
origin  git@github.com:Ouy5517/project4_multi_agent.git (push)
```

4.3 安装 Python 依赖

进入项目目录:

```
cd "C:\Users\ztxw4\Desktop\robocup队内通讯"
```

建议创建虚拟环境：

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
```

安装依赖：

```
python -m pip install pytest matplotlib
```

如果 PowerShell 不允许激活虚拟环境，先执行：

```
Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
```

4.4 解决 Windows 中文/emoji 输出问题

当前 main.py 在 Windows 默认 GBK 控制台下可能因为 emoji 输出报错。每次运行前可以设置：

```
$env:PYTHONUTF8="1"
```

或者执行：

```
chcp 65001
```

推荐用第一种，影响范围更小。

5. 当前项目如何运行

5.1 查看帮助

```
$env:PYTHONUTF8="1"
python main.py --help
```

当前支持的参数包括：

- `--mode mock`：使用项目内置 2D 仿真器。
- `--mode real`：预留给外部仿真器、SDK 或真实机器人接口。
- `--scenario default/pass/shoot/threat`：选择预设场景。
- `--duration`：设置仿真秒数。
- `--headless`：只输出日志，不输出 ASCII 可视化。
- `--export-csv`：导出决策日志 CSV。

5.2 运行最小演示

```
$env:PYTHONUTF8="1"
python main.py --headless --duration 10
```

5.3 运行传球场景

```
$env:PYTHONUTF8="1"
python main.py --scenario pass --duration 30 --export-csv
```

5.4 运行射门场景

```
$env:PYTHONUTF8="1"
python main.py --scenario shoot --duration 30 --export-csv
```

5.5 运行威胁/卡位场景

```
$env:PYTHONUTF8="1"
python main.py --scenario threat --duration 30 --export-csv
```

5.6 运行测试

先安装 pytest:

```
python -m pip install pytest
```

再运行:

```
$env:PYTHONUTF8="1"
python -m pytest -q
```

测试报告需要记录:

- 测试用例名称。
- 测试目标。
- 输入场景。
- 预期结果。
- 实际结果。
- 是否通过。
- 如果失败, 说明原因和修复记录。

6. Git 协作流程

多人开发时, 不建议所有人都直接在 `main` 分支上改。推荐流程:

6.1 每天开始前同步

```
git pull origin main
```

6.2 为自己的任务创建分支

示例：负责传球策略的人创建分支：

```
git checkout -b feature/pass-strategy
```

负责测试的人：

```
git checkout -b test/strategy-tests
```

负责文档的人：

```
git checkout -b docs/course-report
```

6.3 修改后查看状态

```
git status
```

6.4 提交代码

```
git add .  
git commit -m "实现传球策略测试"
```

6.5 推送分支

```
git push -u origin feature/pass-strategy
```

6.6 合并方式

在 GitHub 上创建 Pull Request，由至少 1 名组员检查后再合并到 main。

检查重点：

- 是否能运行。
- 是否有测试。
- 是否影响公共接口。
- 是否更新了文档。
- 是否有无关文件、临时文件、私钥或大文件。

7. 建议的小组分工

作业要求每组 8-15 人，并且每个人要有可追踪的独立贡献。结合当前仓库，建议这样分：

角色	负责内容	对应文件/成果
项目负责人	进度、任务拆分、GitHub PR 管理、答辩协调	会议记录、计划表、答辩串讲

角色	负责内容	对应文件/成果
需求与文档负责人	需求规格说明书、验收标准、AI 使用说明	docs/SRS.md 、总报告
架构与接口负责人	WorldState 、 RobotAction 、接口文档	common/ 、 docs/API.md
传球策略负责人	传球条件、接球区域、传球日志	strategy/strategy_pass.py
带球与射门负责人	带球推进、射门判断	strategy/strategy_dribble.py 、 strategy/strategy_shoot.py
跑位与卡位负责人	支援跑位、防守卡位	strategy/strategy_position.py 、 strategy/strategy_block.py
决策引擎负责人	状态机/行为树、策略切换	decision/decision_fsm.py
测试负责人	单元测试、集成测试、失败案例	tests/ 、测试报告
集成与演示负责人	演示脚本、视频录制、运行说明	main.py 、 README.md 、演示视频

8. 推荐完成路线

第 1 步：把当前仓库跑通

目标：

- 每个组员都能拉取仓库。
- 每个组员都能运行 main.py 。
- 至少一台电脑能运行全部测试。

操作：

```
git pull origin main
python -m pip install pytest matplotlib
$env:PYTHONUTF8="1"
python main.py --headless --duration 10
python -m pytest -q
```

产出：

- 环境配置截图。
- 成功运行截图。
- 首次测试结果。

第 2 步：冻结最小公共接口

目标：

- 明确 WorldState 里必须包含哪些字段。
- 明确 RobotAction 能提供哪些动作。

- 避免后续每个人各写一套数据结构，导致无法集成。

建议最小字段：

```
WorldState:
  time
  ball.position
  ball.velocity
  robots[team][id].position
  robots[team][id].orientation
  robots[team][id].role
  goals.blue
  goals.yellow
  field.width
  field.height

RobotAction:
  move_to(robot_id, target)
  turn_to(robot_id, angle)
  dribble(robot_id, direction, speed)
  kick(robot_id, target, power)
  stop(robot_id)
  reset(robot_id)
```

产出：

- docs/API.md
- docs/WorldState字段表.md
- docs/RobotAction接口说明.md
- 至少 1 个 mock 输入样例。

第 3 步：完成最低验收功能

目标：

- 稳定完成一次有效传球，或一次带球推进。
- 输出可读的决策日志。

建议优先完成传球闭环：

1. WorldState 读取球、己方机器人、对手、球门位置。
2. 判断当前最近球的机器人为持球者。
3. 找到接球队友。
4. 判断接球区域是否安全。
5. 执行 PASS 。
6. 接球队友移动到接球点。
7. 日志记录 CHASE -> PASS -> RECEIVE 或类似状态变化。

验收标准：

- 日志中能看到两个机器人参与。

- 能看到传球开始、传球目标、传球结果。
- 如果失败，要能说明失败原因，例如距离过远、对手拦截、角度不合适。

产出：

- 传球演示命令。
- 传球演示视频。
- 传球场景测试报告。

第 4 步：补齐带球、跑位、卡位和射门

目标：

- 让系统不只是单点传球，而是体现策略切换。

建议策略优先级：

如果球在己方控制范围内：
 如果射门条件好 -> SHOOT
 如果队友位置更好 -> PASS
 否则 -> DRIBBLE
如果球不在控制范围内：
 最近机器人 -> CHASE
 其他机器人 -> POSITION
如果对手威胁球门：
 防守机器人 -> BLOCK

每个策略都应有：

- 输入：WorldState
- 输出：RobotAction
- 日志：策略选择原因
- 测试：至少 2-3 条用例

第 5 步：完成测试体系

课程要求不能只展示成功案例。建议至少准备 15 条测试：

类型	数量	示例
WorldState 测试	2	坐标读取、角色读取
传球策略测试	3	正常传球、距离过远、对手拦截
带球策略测试	2	向前推进、边界保护
射门策略测试	2	可射门、角度不好不射门
跑位/卡位测试	3	支援位、拦截位、威胁检测
状态机测试	2	状态切换、异常回退
集成测试	1-2	完整演示场景

测试命令：

```
$env:PYTHONUTF8="1"
python -m pytest -q
```

第 6 步：准备演示视频

最低要展示：

- 项目启动。
- 场景运行。
- 两个机器人完成传球或带球推进。
- 控制台或 CSV 中出现决策日志。

推荐录制 2-3 分钟：

1. 10 秒展示项目目录和启动命令。
2. 40 秒展示 default 场景。
3. 40 秒展示 pass 场景。
4. 40 秒展示 threat 或 shoot 场景。
5. 20 秒展示日志文件和测试结果。

第 7 步：整理最终文档

作业要求的文档建议目录包括：

1. 项目名称。
2. 项目背景与意义。
3. 需求分析：用户角色、功能需求、非功能需求、验收标准。
4. 系统总体设计：架构图、模块图、数据流和运行流程。
5. 详细设计：核心模块、接口、类/函数、状态机或算法流程。
6. 系统实现：开发环境、关键代码说明、运行方法。
7. 系统测试：测试计划、测试用例、测试结果、异常处理。
8. 项目管理：小组分工、进度计划、会议记录、版本管理。
9. 成果展示：截图、视频说明、实验数据和量化指标。
10. 问题分析与改进方向。
11. 参考资料。

建议在仓库里新建：

```
docs/
  SRS.md
  Design.md
  API.md
  TestReport.md
  MeetingRecords.md
  AIUsage.md
  FinalReport.md
outputs/
  logs/
```

```
csv/  
videos/
```

9. 如何结合 Booster 官方开源项目继续扩展

9.1 参考 RoboCup Demo 的系统分层

Booster RoboCup Demo 官方说明中包含三个程序：

- `vision`：检测机器人、足球和场地，并计算坐标。
- `brain`：读取视觉数据和比赛控制数据，做判断并控制机器人。
- `game_controller`：读取裁判机广播数据，转换成 ROS2 topic。

我们项目可以对应为：

```
vision / 仿真平台 / 其他小组 WorldState -> common/world_state.py  
brain -> decision/ + strategy/  
robot control -> common/robot_action.py + bridge/  
logs -> outputs/
```

9.2 接 ROS2 SDK 的方向

ROS2 SDK 提供机器人控制相关的消息和服务定义，例如底层状态、IMU、低层命令、电机状态、RPC 服务等。我们不需要一开始就使用全部消息，只需要把它看作未来真实接口来源。

建议做一个适配层：

```
bridge/  
  ros2_world_state_adapter.py  
  ros2_robot_action_adapter.py
```

适配层职责：

- 把 ROS2 topic/service 转换成我们统一的 `WorldState`。
- 把我们的 `RobotAction` 转换成 ROS2 控制命令。
- 保持 `strategy/` 和 `decision/` 不依赖 ROS2。

这样做的好处是：即使暂时没有真实机器人，策略代码仍然能在 mock 仿真里测试。

9.3 接 Booster Robotics SDK 的方向

Booster Robotics SDK 支持 C++ 和 Python。官方推荐的 Python 使用方式是安装预构建包：

```
pip install booster_robotics_sdk_python --user
```

如果后续要接真实机器人或官方 SDK，可以在 `bridge/real_robot_action.py` 中实现：

```
RobotAction.move_to -> SDK 移动接口  
RobotAction.turn_to -> SDK 转向接口
```

```
RobotAction.kick -> SDK 踢球/动作接口  
RobotAction.stop -> SDK 停止接口
```

课程阶段不建议把真实 SDK 调用写死在策略里，应统一放在 `bridge/`。

10. WSL2 + Ubuntu 22.04 进阶安装路线

如果要跑 ROS2、官方 Demo 或更接近 Booster 官方环境，按下面做。

10.1 安装 WSL2

在管理员 PowerShell 执行：

```
wsl --install -d Ubuntu-22.04
```

重启电脑后打开 Ubuntu，创建 Linux 用户。

10.2 更新 Ubuntu

```
sudo apt update  
sudo apt upgrade -y  
sudo apt install -y git curl wget build-essential cmake python3 python3-pip  
python3-venv
```

10.3 安装 ROS2 Humble

官方 ROS2 Humble 安装步骤较长，建议按 ROS2 官方文档执行。核心安装完成后，需要能执行：

```
source /opt/ros/humble/setup.bash  
ros2 --version
```

常用开发工具：

```
sudo apt install -y python3-colcon-common-extensions python3-rosdep
```

10.4 克隆 Booster 官方相关仓库

建议在 WSL 中建立单独目录，不要直接混进课程仓库：

```
mkdir -p ~/booster_ws/src  
cd ~/booster_ws/src  
git clone https://github.com/BoosterRobotics/booster_robotics_sdk.git  
git clone https://github.com/BoosterRobotics/booster_robotics_sdk_ros2.git  
git clone https://github.com/BoosterRobotics/robocup_demo.git
```

如果 GitHub 网络不稳定，可以先只下载压缩包查看代码，不影响我们课程项目先推进。

10.5 构建 ROS2 SDK

```
cd ~/booster_ws
source /opt/ros/humble/setup.bash
colcon build
source install/setup.bash
```

10.6 RoboCup Demo 参考使用

RoboCup Demo 官方提供了无 CUDA 和有 CUDA 两种构建脚本：

```
./scripts/build_no_cuda.sh
./scripts/build.sh
```

虚拟机器人运行脚本：

```
./scripts/sim_start.sh
```

实机运行脚本：

```
./scripts/start.sh
```

注意：这一步是进阶参考，不是完成题目四最低验收的前置条件。

11. 风险与规避

风险	表现	规避办法
Git 权限没配好	不能 pull/push	先确保 <code>ssh -T git@github.com</code> 成功
依赖缺失	No module named pytest	安装 <code>pytest matplotlib</code>
Windows 编码问题	emoji 输出报错	设置 <code>\$env:PYTHONUTF8="1"</code>
公共接口频繁变动	各模块无法集成	第 2 阶段冻结最小字段，只做兼容扩展
只演示成功案例	测试报告不合格	加入失败案例和异常处理说明
直接复制官方 Demo	解释不清楚个人贡献	只参考架构，核心策略和文档自己实现
没有过程记录	分工不可追踪	每人用 Git 分支、提交记录、会议记录证明贡献

12. 最终提交检查清单

代码类：

- ☐ `main.py` 可运行。
- ☐ `--scenario pass` 可展示传球或接球判断。
- ☐ `--scenario threat` 或等价场景可展示卡位/防守。
- ☐ `--export-csv` 能输出日志。
- ☐ `python -m pytest -q` 能通过。

文档类：

- ☐ README 说明环境配置、运行方法、演示步骤。

- ☐ 需求规格说明书。
- ☐ 概要设计说明书。
- ☐ 详细设计说明书。
- ☐ 接口说明文档。
- ☐ 测试报告，包含失败案例。
- ☐ 小组分工表。
- ☐ AI 使用说明。
- ☐ 项目总结报告。

演示类：

- ☐ 至少一个核心功能演示视频。
- ☐ 决策日志截图或 CSV。
- ☐ 测试结果截图。
- ☐ 答辩 PPT。

13. 参考资料

- Booster 官方开源资料：<https://www.booster.tech/cn/open-source/>
- Booster Robotics SDK：https://github.com/BoosterRobotics/booster_robotics_sdk
- Booster ROS2 SDK：https://github.com/BoosterRobotics/booster_robotics_sdk_ros2
- Booster RoboCup Demo：https://github.com/BoosterRobotics/robocup_demo
- 当前课程仓库：https://github.com/Ouy5517/project4_multi_agent